أكاديمية طويق
Tuwaiq Academy

13 AUGUST 2026

GPU-ACCELERATED

Machine learning Report
Predict accident severity

WRITTEN BY

Afnan Alshalahi

INSTRUCTOR

Banan ALnemri

COURSE

Scalable Data Science with Python

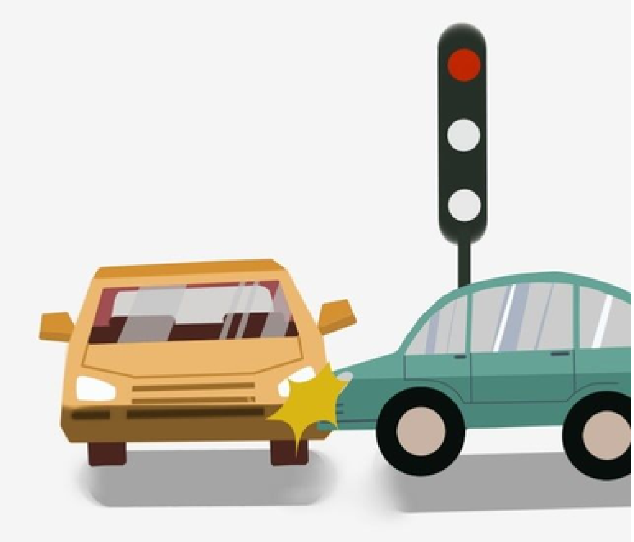
TABLE OF CONTENTS

1-Abstract

Using machine learning, this project predicted the severity of traffic accidents and handled the data with GPUs. We worked with a US traffic accident dataset with seven million records and picked a million of them for ML experiments and predictions. The data was prepped and cleaned with RAPIDS cuDF by converting data types, filling in missing values, dropping unnecessary columns, and creating new features to boost the results. Three models were trained on GPU: logistic regression, random forest, and KNN, and we checked their performance based on accuracy, overall F1-score, weighted F1-score, training time, and inference time.

The highest accuracy went to the random forest at 72.84% and the highest F1-score at 71.45%, while KNN achieved the highest overall F1-score at 46.37%. The GPU was monitored during the workflow, and KNN ended up using the GPU more on average.

2-Introduction



The field of data analysis is developing rapidly, and when machine learning came into play, it became a powerful force in producing analysis and prediction models that greatly help the growth of various fields.
As for individual safety, traffic accidents have become one of the main problems that governments face and have a negative impact, making it necessary to put plans, programs, and efforts in place to reduce such issues.
Predicting using machine learning models has become an essential part of decision-making and identifying the causes and factors leading to such accidents.In this project, machine learning was used to predict the severity of traffic accidents based on several factors, including weather conditions, traffic signals, time, accident location, and others.

As the amount of data grows day by day, it becomes difficult to process using the CPU, so the RAPIDS system was used to perform processing using CUDF and CUML. The GPU processor accelerates the project by comparing the three models and analyzing their predictive performance, training time, and memory usage.

3-Problem Statement

Studies point to multiple factors causing traffic accidents, including geographic location, weather, timely emergency response, road characteristics, and more. The challenge lies in measuring the severity of an accident, as determining severity can speed up quick response, improve traffic flow, and aid road safety planning.

The dataset contains unbalanced severity levels, making it harder to predict the less frequent severity levels. In this project, severity levels were classified into Severity 1, Severity 2, Severity 3, and Severity 4. To describe the accident, numerical and logical data were used to cover the accident, location, weather, and time.

Problem statement: Predicting the accident severity level considering factors like traffic characteristics, weather conditions, geographic location, and timely access was done using machine learning models on a GPU.

4-Objectives

1. Analyzing and processing a traffic accident dataset containing 1,000,000 traffic accident records using RAPIDS cuDF and speeding up the process through GPU.
2. Organizing, cleaning, and preparing the data for machine learning by detecting missing values, removing unimportant variables, converting data types, and creating time-related features.
3. Using three models with GPU acceleration via cuML RAPIDS: logistic regression, random forest, and KNN, and evaluating their performance using accuracy, overall and weighted F1-score, GPU memory usage, training time, and comparing the three models.
4. Identifying the most important features closely related to accident severity and analyzing the balance between prediction performance and GPU resource utilization.

5-Data

4.1-Data Source

The dataset used in this project is the U.S. traffic accidents dataset from the OpenML repository. It has seven million records and includes detailed information on accident severity, weather at the time of the accident, geographic location, road characteristics, and time-related variables.
As mentioned earlier, the data contains many records of incidents, which made it the best set for a large machine learning project and for trying out environments like GPU. Due to limited available memory, only one million incident records were used for analysis, processing, testing the models, and measuring them.

The openML repository provided the data in ARFF format, and it was processed using RAPIDS. cuDF was used to load, process, and transform the data, and then to train and evaluate machine learning on it.
Dataset Source: OpenML - US Accidents dataset
OpenML Dataset - US Accidents

4.2-Dataset Description

One million traffic accident records were targeted in this project from the dataset in the United States. After the initial cleaning process and removing irrelevant columns, the number of columns was reduced to 39 while keeping the main dataset of one million records. Additional features based on time were created, bringing the total to 48 columns. The target variable is the severity of the traffic accident, which has four multiple categories: Severity 1, Severity 2, Severity 3, and Severity 4. The distribution of the target variable was as follows:

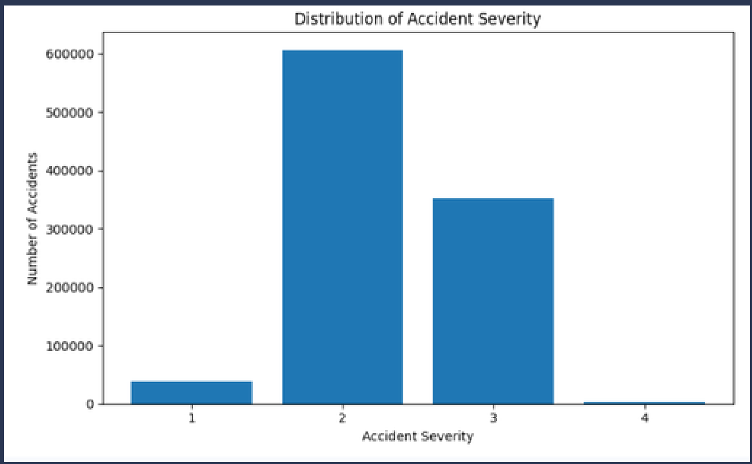
Severity	Number of Records
1	38,381
2	606,235
3	352,187
4	3,197
Total	1,000,000

We ended up with 28 features in the final matrix used for machine learning. These included logical variables like road characteristics, traffic, traffic signals, bumps, railways, and others, as well as numerical variables like latitude, longitude, temperature, weather, wind speed, rainfall, accident duration, hour, day, month, and other numerical variables related to the data subject. All missing value rows were filled, and the target distribution was imbalanced, with severity 2 taking up a large portion in the biggest category and severity 4 in the smallest category. This variation and difference were taken into account when interpreting model results, focusing on the overall F1 score and retrieving each category individually.

4.3-Exploratory Data Analysis

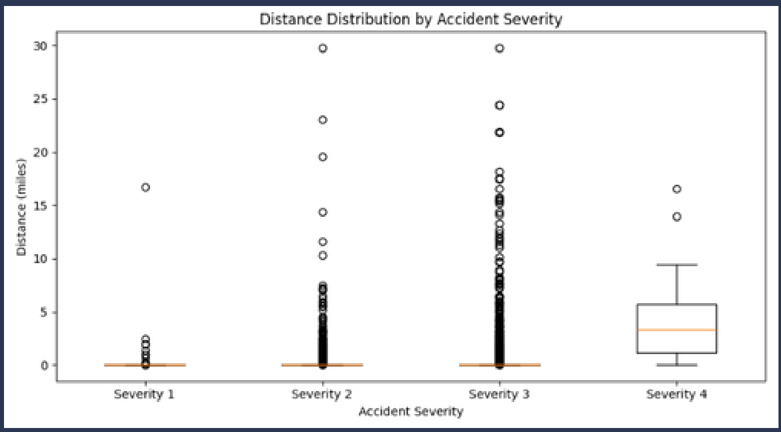
An exploratory analysis was conducted to understand the structure of the data and how the target variable is distributed, as well as to see if there are patterns affecting the machine learning process. The analysis was carried out on the data using cuDF on the GPU, while the CPU was used to transfer small results for visualization. During the analysis, the distribution of accident severity was examined. The results showed a clear imbalance in the class distribution. Severity 2 was the most common class with 606,235 records, followed by severity 3 with 352,187 records. Severity 1 had 38,381 records, while severity 4 had 3,197 records. These results are important because they show the imbalance, explaining how a model can achieve high overall accuracy while performing poorly on the less frequent classes.

Figure 1. Distribution of Traffic Accident Severity



In the second analysis, the distribution of accident distance across the four severity categories was studied. Only a sample of 10,000 records was used to save on memory and avoid process freezing while keeping the full dataset for modeling. The box plot allows for a comparison of distance distributions between severity levels and helps identify differences and outliers.

Figure 2. Distance distribution by accident severity



Overall, the exploratory data analysis showed that the dataset has a significant imbalance between categories, and the numerical variables related to accidents might have different distributions depending on the severity levels. These observations were taken into account during the preprocessing and model evaluation stages. The analysis clarified that there's a big gap in distribution between categories, and numerical variables related to the accident may vary in distribution according to the severity level. These points were considered during both the preprocessing and evaluation phases.

4.4-Data Preprocessing

During the data preparation stage, several steps were applied, making sure that the number of records was not less than one million. The data was processed using cuDF with a GPU. As for the missing values, they were examined, and some columns containing them were found, including those related to weather, temperatures, humidity, cold feeling, wind speed, lack of visibility, and pressure. These were handled using the median since it is the least affected by outliers if they appear later, and some were converted to numeric values. The rain feature contained a non-numeric value called "Fair," and this feature was converted into a numeric value. Regarding some invalid values, they were treated as missing before processing. In the visibility feature, the data type was converted to a numeric value before being used as a machine learning feature. Before moving on to the next step, it was ensured that all weather variables had no missing values that could affect workflow.

The dataset was searched and examined to ensure there were no duplicate rows, and several unnecessary columns with missing values were deleted from the analysis. The number of columns went from 44 to 39. To capture temporal patterns, the start and end times of incidents were changed from object/text format to a format that includes both date and time. The number of time-based features was also increased to understand the severity of the incident, including hour, day, month, day of the week, and whether it was on a weekend or not.

The duration of the incident was calculated in minutes using the difference between the end time and the start time. Some outlier values were discovered during the initial analysis, including the duration, where one record lasted 24 hours and some even longer, which didn't fit the data distribution. To address this, the upper limit for duration was set at the 99th percentile, which was 150.02 minutes.

This helped reduce the maximum duration while ensuring all data was retained without any loss or damage. In the end, when selecting modeling features, it was ensured they were separated from the target variable, severity. The final features included 28 features without any missing values or duplicates. The numeric features were scaled before using them in the machine learning stage, and the binary traffic-related variables were retained as binary features.

4.5-Train / Validation / Test Split

During the training phase, the data was split into training and testing sets using an 80/20 method. As is known, the training set contains 800,000 records while the testing set contains 200,000 records. The training data was used to train the models, and the test data was kept aside to be used later only for the final evaluation of the models. The split was done using a fixed random state of 42 to ensure we could replicate the experiment. The target variable is severity, and we have 28 features. A separate validation set wasn't created because wide hyperparameter tuning wasn't done in this basic experiment. Instead, the same training and test sets were used each time to compare the three models to ensure fair evaluation for all models, so each model was trained and evaluated using the same data split.

The resulting datasets were:

Dataset	Records	Features
Training	800,000	28
Testing	200,000	28
Total	1,000,000	28

6-Methodology and Training

5.1-Models Considered

Three AI models were chosen to assess the problem of classifying traffic accident severity in the United States, using GPU support with RAPIDS cuML. 1- Logistic Regression: It was used in the evaluation process because it provides an excellent approach to classification and allows identifying a baseline for comparison with other models. 2- Random Forest: This model is known for capturing nonlinear relationships between accident features and their severity, and it was used to identify the most important features that contributed to predictions in the model. 3- K-Nearest Neighbors (KNN): Since the data contains many numerical and measured features, the KNN model provides comparisons that help support the other two models, and it was used as a distance-based classification method.

5.2Training Setup

This project was done on the GOOGLE COLAB environment using the RAPIDS ecosystem, relying on cuDF for fast data processing on the GPU and cuML. An NVIDIA Tesla T4 GPU with 15 GB was used. After the final cleaning and analysis, we ended up with 28 features divided into 800,000 records for training and 200,000 records for testing. For a fair comparison, the same split was used for all models. The size of numeric features was reduced before training.

This project was done on the GOOGLE COLAB environment using the RAPIDS ecosystem, relying on cuDF for fast data processing on the GPU and cuML. An NVIDIA Tesla T4 GPU with 15 GB was used. After the final cleaning and analysis, we ended up with 28 features divided into 800,000 records for training and 200,000 records for testing. For a fair comparison, the same split was used for all models. The size of numeric features was reduced before training.

Model Main Configuration Logistic Regression cuML Logistic Regression with scaled features, Random Forest 100 estimators, maximum depth = 16, random state = 42
KNN cuML K-Nearest Neighbors using the scaled feature matrix.

Training times were measured using time.perf_counter(), and during that, GPU and memory usage were monitored using NVIDIA-smi while the model was making predictions. This allowed us to compare the computational performance and predictive performance of the three models. The target variable is Severity, which consists of four categories from 1 to 4. The models were evaluated on a test set of 200,000 records using accuracy, Macro F1-score, and Weighted F1-score.

5.3-Hyperparameter Tuning

No extensive parameter tuning was done in this project. Our goal with this experiment was to compare the performance of the three models supported by the GPU on the same dataset and with the same train-test split. For the Random Forest model, we picked a fixed number of 100 estimators, a depth of 16, and 42 random states. The evaluation process included all three models on the same test set, allowing us to compare predictive performance and computational efficiency under consistent conditions. For future optimization, we can use grid search or random search to improve model parameters, especially for Random Forest and KNN.

5.4-Baseline

The basic model assigned to this project was logistic regression because it simply provides a computationally efficient approach for multi-class classification. This allowed us to make it the benchmark for other models and compare them. The logistic regression model achieved an accuracy of 63.87%, a macro F1 score of 29.74%, and a weighted F1 score of 59.38%. These results provided a baseline to assess whether other models could improve classification performance or not. When we compare to the baseline, we see that the random forest performed better with an accuracy of 72.84% and a weighted F1 score of 71.45%, while the KNN model achieved an accuracy of 67.11% and the highest macro F1 score among the three models at 46.37%.

7-valuation and Results

6.1-Evaluation Metrics

The evaluation of the three models was based on accuracy, macro F1 score, and weighted F1 score. The reason for choosing these metrics is that the target variable has four categories of severity in risk, and they are unevenly balanced.
Accuracy measures the proportion of correctly classified incidents out of all test records, and it gives an overall view of the model's performance.
The F1 score calculates the macro F1 for each risk category independently and then takes the average. This metric gives equal importance to all categories, which makes it especially useful for this project since severity 4 is much less common than severity 2. The weighted F1 ratio provides a metric that reflects the overall performance of the models while taking the category distribution into account.

6.2-Results Table

The three GPU-accelerated machine learning models were evaluated using the same 200,000-record test set. The results are summarized in Table 1.

Model	Accuracy	Macro F1	Weighted F1	Training Time (s)	Inference Time (s)
Logistic Regression	0.6387	0.2974	0.5938	5.711	0.774
Random Forest	0.7284	0.4111	0.7145	8.567	2.566
KNN	0.6711	0.4637	0.6658	0.067	9.675

Table 1: Comparison of predictive and computational performance among the three models

The model that achieved the best and highest accuracy at 72.84% and the highest weighted F1 score at 71.45% is the Random Forest, making it the strongest model in terms of overall predictive performance. Meanwhile, the KNN model achieved the highest macro F1 score at 46.37%, showing that it performed best on average across the four severity classes, even though its accuracy was lower than that of the Random Forest.

As for the Logistic Regression model, it had the lowest performance among the models, with an accuracy of 63.87% and a macro F1 score of 29.74%. In terms of the shortest training time, KNN was the best at just 0.067 seconds, but it had the longest inference time at 9.675 seconds, while the Logistic Regression model had an inference time of 0.774 seconds, and the Random Forest took 2.566 seconds for inference.

6.3-GPU Performance Analysis

Model	Averag GPU Utilization	Maximum GPU Utilization	Maximum GPU memory
Logistic Regression	0.00%	0.00%	927 MiB
Random Forest	0.00%	0.00%	953 MiB
KNN	97.96%	100%	971 MiB

Table 2. GPU performance metrics recorded during model inference.

The GPU monitoring results showed clear differences between the three models. KNN had the highest GPU utilization, reaching an average of 97.96% and a maximum of 100%. Logistic Regression and Random Forest showed approximately 0% average utilization during the short inference measurements. GPU memory usage remained relatively low for all three models, ranging from 927 MiB to 971 MiB.

6.4-Error Analysis

The three models and the way they interpreted the severity levels weren't consistent and all of them were on the same rhythm. In the random forest model, we saw how it achieved the best accuracy, but it wasn't consistent across all categories because the dataset itself wasn't very balanced. After analyzing the confusion matrix and the classification report, it showed that the model correctly classified most of the incidents in category 2, while for smaller categories like category 4, it became hard to predict.

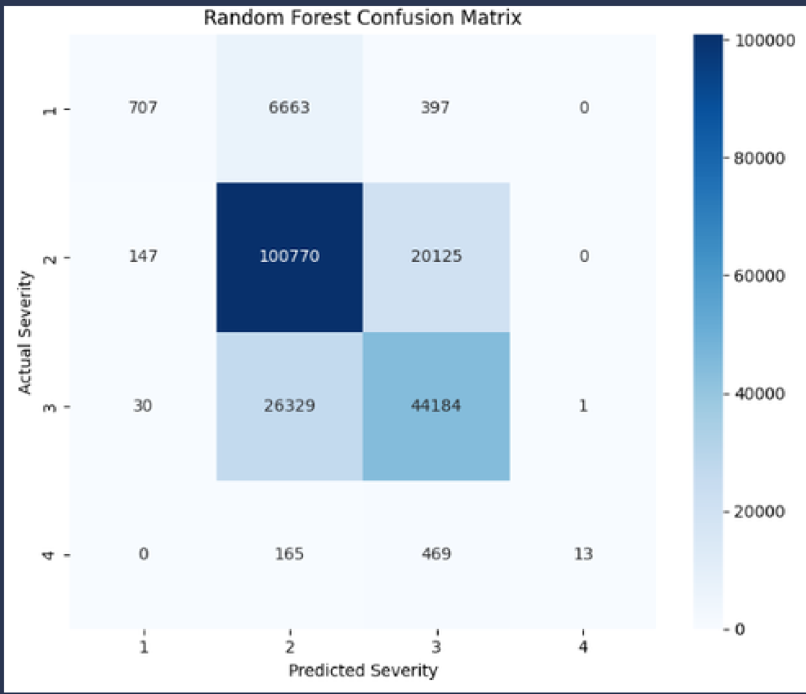


Figure 3: Confusion matrix for the random forest model.

The results indicated that Severity 2 achieved the highest recall rate of 83.25%, which showed that the model works better on the larger category. Meanwhile, Severity 4 recorded only 2.01%, and this big difference showed that the model only recognized a small number of severe cases. The reason is mostly due to the lack of Severity 4 examples in the training data. Error analysis showed that overall accuracy alone is not enough to assess issues with unbalanced categories in the Random Forest model, even though it delivered the best performance. Providing the overall F1 rate and recall for each category gave a more complete picture of the model's performance across all severity levels.

6.5-Feature Importance / Interpretation

In the random forest model, feature importance was used to determine which features contributed the most to better model predictions. The analysis was carried out using the feature importance values provided by the trained random forest model.

The ten most important features were:

Rank	Feature	Importance
1	Weekday	0.1513
2	Start_Lng	0.1281
3	Start_Lat	0.1212
4	Roundabout	0.0804
5	Bump	0.0617
6	Amenity	0.0545
7	Wind_Chill(F)	0.0524
8	Distance(mi)	0.0495
9	Give_Way	0.0411
10	Pressure(in)	0.0407

The three most important features in the random forest model were the days of the week, longitude, and latitude, and the importance of longitude and latitude may indicate that the severity of an accident can vary based on location. Time-related information like the days of the week can significantly contribute to accident severity. Road-related variables like traffic priority and speed bumps emerged among the key features. Meanwhile, weather-related variables like cold winds and pressure had little impact on the model. It's important to keep in mind that feature importance reflects their usefulness for the model's predictions, not necessarily that they directly cause accident severity.

8-Challenges

Of course, there are several challenges faced in the project related to how to handle big data and deal with different types of data like time and date, as well as measuring the performance of the GPU.

Challenge 1: Handling data on a large scale
Working with a million data records is challenging and requires careful memory and computing management. So, RAPIDS cuDF was used to load and preprocess the data with GPU acceleration.

Challenge 2: Converting Date, Time, and Invalid Values
The Start_Time and End_Time columns were entered as text/object data. So, they were converted to datetime format using cuDF, which caused errors because some values weren't directly compatible with the datetime format. The columns were then cleaned up and converted, and the incident duration was calculated along with creating additional time-based features.

Challenge 3: Imbalance of target groups
The target goal was intensity, and of course, with its different levels, it became unbalanced. He pointed out that intensity 2 represents the majority of the records, while intensity 4 represents the least, with only 3,197 records. This led to calling upon the smaller categories, especially intensity 4. So, the average F1-score and class-specific metrics were made to provide a more comprehensive evaluation of the models.

Challenge 4: Differences in GPU Usage Between Models
During the work with the three models, they differed in how they used the GPU during inference. The Random Forest and Logistic Regression models showed almost no GPU usage, around 0%. Meanwhile, the KNN model used the GPU at 97.96%. All of them used the GPU, but in different ways.

9-Future Work

The model is useful and provides an excellent comparison in terms of machine learning using GPU units, and it adapts well to future improvements. Hyperparameter tuning: Grid Search or Random Search can be applied to optimize parameters like the number of trees and maximum depth in Random Forest, and the number of neighbors in KNN. This can improve predictive performance, especially for less represented severity categories. Handling imbalance between severity categories: We can make the categories weighted equally or use augmentation methods to improve predictions for lower-frequency categories like severity 1 and severity 4. Feature engineering and interpretation: It's possible to create additional features related to weather conditions, geographic areas, time periods, and road characteristics. More advanced interpretation methods can also be used to get a better understanding of how these features affect accident severity predictions.

10-Discussion

The results showed that the three models performed differently on the traffic accident severity problem. The Random Forest model generally gave the best result, with an accuracy of 72.84% and a weighted F1 score of 71.45%. Meanwhile, the KNN model had a lower accuracy of 67.11% but achieved the highest overall F1 score of 46.37%. This means that KNN performed better when looking at the four severity categories more evenly, while Random Forest performed better overall.

One important issue in the results was the imbalance between severity categories. Most accidents were severity 2, while severity 4 had only 3,197 records. Because of this, the Random Forest model was very good at predicting severity 2, with a recall rate of 83.25%, but had a very low recall for severity 4 at just 2.01%. This shows why accuracy alone isn't enough to evaluate a model. A model can have good overall accuracy but still struggle to predict less common categories.

GPU results also varied between the models. KNN had the highest GPU usage, averaging 97.96%, while logistic regression and random forest showed around 0% average usage during the measured inference period. KNN also took much longer during inference, showing that higher GPU usage doesn't necessarily mean the model is better or faster.

There were some limitations in this project. We used 1,000,000 records instead of the full dataset due to available computing resources. We also didn't do thorough hyperparameter tuning or use specific techniques to balance severity classes. Another limitation is that the dataset is based on traffic incidents in the United States, so the results might not directly apply to other countries.

Overall, the project showed that GPU-supported tools can be useful when working with a large traffic accident dataset. The random forest model was the best in terms of overall accuracy and weighted F1 scores, while the KNN model performed better in overall F1 and used the GPU much more. In the future, improving predictions for the less common categories will be an important step, especially for severity category 4, since correctly identifying severe accidents can be more important than just achieving high overall accuracy.

11-References

[1] Moosavi, S., Samavatian, M. H., Parthasarathy, S., & Ramnath, R. (2019). A Countrywide Traffic Accident Dataset. arXiv. Paper link □

[2] OpenML. (n.d.). US Accidents Dataset, Dataset ID 46650. OpenML. OpenML dataset □

[3] NVIDIA RAPIDS. (2026). cuDF Documentation. RAPIDS Documentation. cuDF Documentation □

[4] NVIDIA RAPIDS. (2026). cuML Documentation. RAPIDS Documentation. cuML Documentation □

[5] NVIDIA. (2026). RAPIDS API Documentation. RAPIDS Documentation □